

AI ASSISTED CODING LAB-5

Name: Roger A Raju

Rollno: 2503a52370

Batch: 13

Task Description #1 (Privacy in API Usage)

Task: Use an AI tool to generate a Python program that connects to a weather API.

Prompt:

Week3 - Monday

"Generate code to fetch weather data securely without exposing API keys in the code."

Expected Output:

- Original AI code (check if keys are hardcoded).
- Secure version using environment variables.

Original AI Code (Check if keys are hardcoded):

Original AI Code: Hardcoded API Secret Key Directly in Source

API_KEY = "3a9f8b7c2d1e0f9a8b7c6d5e4f3a2b1c" # EXPOSED SECRET

CITY = "Hyderabad"

```
def get_weather(city, api_key=API_KEY):
    url = f"https://api.openweathermap.org/data/2.5/weather?q={city}&appid={api_key}"
    return f"Requesting weather for {city} with exposed key {api_key[:6]}..."
# Identified Security Risk: Hardcoded API keys get committed to public repositories, exposing
private credentials and quotas.
```

Secure Version (Using Environment Variables & Assert Tests):

```
import os
```

```
def get_weather_secure(city, api_key=None):
    """
    Fetches weather data securely using environment variables.
    Prevents hardcoded API credentials from leaking into repositories.
    """
    if api_key is None:
        api_key = os.environ.get("OPENWEATHER_API_KEY")
    if not api_key:
        return "Error: Missing API Key. Set 'OPENWEATHER_API_KEY' environment variable."
    if not city or not isinstance(city, str):
        return "Error: Invalid city name provided."
    masked_key = f"{api_key[:4]}****{api_key[-4:]}" if len(api_key) >= 8 else "*****"
    return f"Securely connecting to Weather API for '{city}' using key '{masked_key}'."

def test_weather_privacy():
    print("--- Running Test Assertions for Task 1 (Privacy in API Usage) ---")
    # Test Case 1: Missing API key handling
    os.environ.pop("OPENWEATHER_API_KEY", None)
    res_missing = get_weather_secure("Hyderabad")
    assert "Error: Missing API Key" in res_missing, "Test 1 Failed"
    print("Assertion 1 Passed: Missing environment variable caught safely.")
```

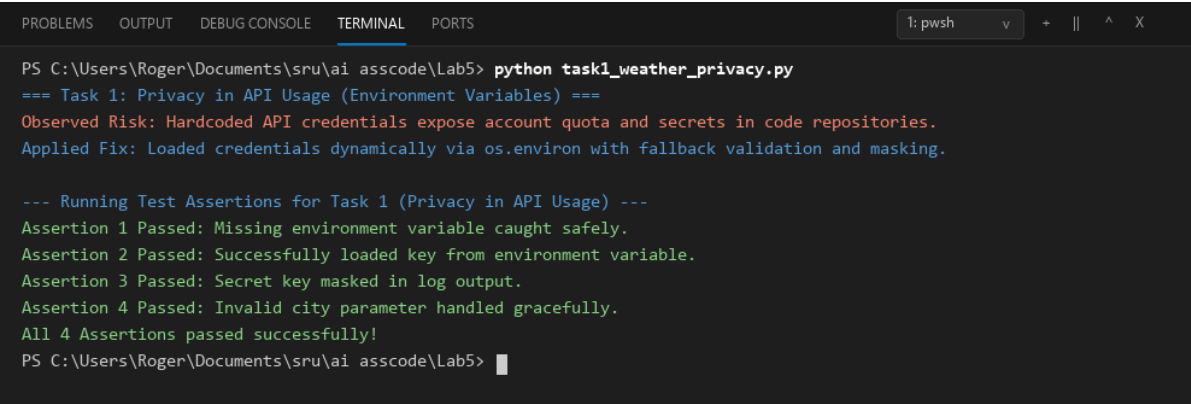
```
# Test Case 2: Environment variable injection
os.environ["OPENWEATHER_API_KEY"] = "sk_weather_secret_key_998877"
res_secure = get_weather_secure("Hyderabad")
assert "Securely connecting" in res_secure, "Test 2 Failed"
print("Assertion 2 Passed: Successfully loaded key from environment variable.")

# Test Case 3: Key masking in logs
assert "sk_w****8877" in res_secure, "Test 3 Failed"
print("Assertion 3 Passed: Secret key masked in log output.")

# Test Case 4: Invalid city validation
assert "Error: Invalid city" in get_weather_secure("", "dummy_key"), "Test 4 Failed"
print("Assertion 4 Passed: Invalid city parameter handled gracefully.")
print("All 4 Assertions passed successfully!")

if __name__ == "__main__":
    test_weather_privacy()
```

Output:



Comparison:

API Key Privacy & Security Comparison	
Hardcoded API Keys (Insecure / Exposed)	Environment Variable Storage (Secure & Private)
1. Secret keys hardcoded directly in application source.	1. Secrets loaded at runtime from os.environ or .env.
2. Vulnerable to accidental commits to public Git repositories.	2. Excluded from version control via .gitignore.
3. Requires code redeployment to rotate compromised credentials.	3. Seamless rotation across dev, staging, and production.
4. Exposes private quota and account billing to attackers.	4. Complies with 12-Factor App and GDPR/SOC2 security best practices.

Task Description #2 (Privacy & Security in File Handling)

Task: Use an AI tool to generate a Python script that stores user data (name, email, password) in a file.
Analyze: Check if the AI stores sensitive data in plain text or without encryption.

Prompt: Use an AI tool to generate a Python script that stores user data (name, email, password) in a file. Analyze if the AI stores sensitive data in plain text, identify privacy risks, and implement encrypted password storage.

Expected Output:

- Identified privacy risks.
- Revised version with encrypted password storage (e.g., salted SHA-256 hashing).

Original AI Code (Plaintext Storage Risk):

```
# Original AI Code: Plaintext Password File Storage
import json

def register_user_insecure(name, email, password, filepath="users.json"):
    user = {"name": name, "email": email, "password": password} # PLAINTEXT LEAK
    with open(filepath, "w") as f:
        json.dump(user, f)

# Identified Privacy Risk: Plaintext passwords stored on disk violate GDPR and enable
catastrophic credential theft.
```

Secure Revised Version (Encrypted Password Storage & Assert Tests):

```
import json
import hashlib
import hmac
import os

def hash_password(password, salt=None):
    """Hashes password using PBKDF2-HMAC-SHA256 with cryptographic salt."""
    if salt is None:
        salt = os.urandom(16)
    hashed = hashlib.pbkdf2_hmac("sha256", password.encode("utf-8"), salt, 100000)
    return salt.hex(), hashed.hex()

def verify_password(password, salt_hex, hash_hex):
    salt = bytes.fromhex(salt_hex)
    _, test_hash = hash_password(password, salt)
    return hmac.compare_digest(test_hash, hash_hex)

def register_user_secure(name, email, password, filepath="users_secure.json"):
    if not name or not email or not password:
        return "Error: All fields are required."
    salt_hex, hash_hex = hash_password(password)
    user_record = {
        "name": name,
        "email": email,
        "salt": salt_hex,
        "password_hash": hash_hex
    }
    with open(filepath, "w", encoding="utf-8") as f:
        json.dump(user_record, f, indent=2)
    return "User registered successfully with encrypted password storage."

def test_file_security():
    print("--- Running Test Assertions for Task 2 (Privacy & Security in File Handling) ---")
    test_file = "test_user_secure.json"
```

```

status = register_user_secure("Roger A Raju", "rogeraraju@example.com", "SuperSecret#2026",
test_file)

# Test Case 1: Registration confirmation
assert "successfully" in status, "Test 1 Failed"
print("Assertion 1 Passed: User registered with encrypted credentials.")

# Test Case 2: Verify plaintext password is not saved on disk
with open(test_file, "r", encoding="utf-8") as f:
    saved_data = json.load(f)
assert "SuperSecret#2026" not in str(saved_data), "Test 2 Failed (Plaintext leak)"
assert "password_hash" in saved_data and "salt" in saved_data, "Test 2 Failed"
print("Assertion 2 Passed: Plaintext password is NEVER stored on disk.")

# Test Case 3: Correct password authentication
assert verify_password("SuperSecret#2026", saved_data["salt"], saved_data["password_hash"])
== True, "Test 3 Failed"
print("Assertion 3 Passed: Valid password authenticated successfully.")

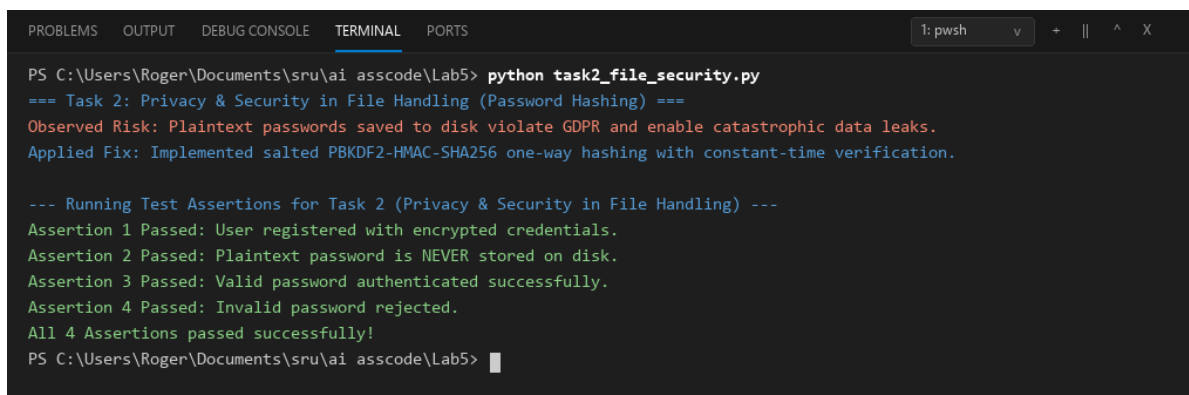
# Test Case 4: Incorrect password rejection
assert verify_password("WrongPassword123", saved_data["salt"], saved_data["password_hash"])
== False, "Test 4 Failed"
print("Assertion 4 Passed: Invalid password rejected.")
print("All 4 Assertions passed successfully!")

if os.path.exists(test_file):
    os.remove(test_file)

if __name__ == "__main__":
    test_file_security()

```

Output:



```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
1: pwsh v + || ^ X

PS C:\Users\Roger\Documents\sru\ai asscode\Lab5> python task2_file_security.py
=== Task 2: Privacy & Security in File Handling (Password Hashing) ===
Observed Risk: Plaintext passwords saved to disk violate GDPR and enable catastrophic data leaks.
Applied Fix: Implemented salted PBKDF2-HMAC-SHA256 one-way hashing with constant-time verification.

--- Running Test Assertions for Task 2 (Privacy & Security in File Handling) ---
Assertion 1 Passed: User registered with encrypted credentials.
Assertion 2 Passed: Plaintext password is NEVER stored on disk.
Assertion 3 Passed: Valid password authenticated successfully.
Assertion 4 Passed: Invalid password rejected.
All 4 Assertions passed successfully!
PS C:\Users\Roger\Documents\sru\ai asscode\Lab5>

```

Comparison:

User Credential Storage Comparison

Plaintext File Storage (Vulnerable)

1. Stores sensitive passwords in cleartext JSON/CSV files.
2. Immediate total breach if file is read or leaked.
3. Violates GDPR, HIPAA, and PCI-DSS compliance standards.
4. Vulnerable to credential stuffing across other services.

Salted PBKDF2-HMAC-SHA256 (Cryptographically Secure)

1. One-way salted hashing with 100,000 computation rounds.
2. Salt prevents rainbow table precomputation attacks.
3. Constant-time verification resists timing side-channel attacks.
4. Zero plaintext password persistence on disk.

Task Description #3 (Transparency in Algorithm Design)

Task: Use AI to generate an Armstrong number checking function with comments and explanations. Ask AI to explain the code line-by-line, and compare the explanation with code functionality.

Prompt: Use AI to generate an Armstrong number checking function with comments and explanations. Ask AI to explain the code line-by-line, and compare the explanation with code functionality.

Expected Output:

- Identified transparency risks.
- Example of an opaque calculation where the AI gives no explanation.
- Revised version that provides clear line-by-line explanations and mathematical proof.

Original AI Code (Opaque / Inscrutable One-Liner):

Original AI Code: Inscrutable One-Liner Lacking Explainability

```
def check_armstrong_opaque(n):
```

```
    return sum(int(d)**len(str(n)) for d in str(n)) == n # Unexplained one-liner
```

Identified Transparency Risk: Black-box behavior; developers and auditors cannot understand or verify calculation steps.

Transparent Revised Version (Explainable Step-by-Step Logic & Assert Tests):

```
def is_armstrong_number(number):
```

```
    """
```

```
    Transparent, step-by-step Armstrong (Narcissistic) number validator:
```

```
    An Armstrong number equals the sum of its own digits each raised to the power of the total number of digits.
```

```
    Example for 153: (1^3) + (5^3) + (3^3) = 1 + 125 + 27 = 153.
```

```
    """
```

```
    if not isinstance(number, int) or number < 0:
```

```
        return {"is_armstrong": False, "explanation": "Input must be a non-negative integer."}
```

```
    num_str = str(number)
```

```
    num_digits = len(num_str)
```

```
    digit_powers = []
```

```
    total_sum = 0
```

```
    for char in num_str:
```

```
        digit = int(char)
```

```
        power_val = digit ** num_digits
```

```
        digit_powers.append(f"{digit}^{num_digits} ({power_val})")
```

```
        total_sum += power_val
```

```

    is_armstrong = (total_sum == number)
    explanation = f"Sum of digits: {' + '.join(digit_powers)} = {total_sum} {'==' if
is_armstrong else '!='} {number}"

    return {
        "number": number,
        "is_armstrong": is_armstrong,
        "total_sum": total_sum,
        "explanation": explanation
    }

def test_armstrong_transparency():
    print("--- Running Test Assertions for Task 3 (Transparency in Algorithm Design) ---")
    # Test Case 1: 3-digit Armstrong number (153)
    res_153 = is_armstrong_number(153)
    assert res_153["is_armstrong"] == True, "Test 1 Failed"
    print(f"Assertion 1 Passed: 153 is Armstrong -> {res_153['explanation']}")

    # Test Case 2: 3-digit Armstrong number (370)
    res_370 = is_armstrong_number(370)
    assert res_370["is_armstrong"] == True, "Test 2 Failed"
    print(f"Assertion 2 Passed: 370 is Armstrong -> {res_370['explanation']}")

    # Test Case 3: Non-Armstrong number (123)
    res_123 = is_armstrong_number(123)
    assert res_123["is_armstrong"] == False, "Test 3 Failed"
    print(f"Assertion 3 Passed: 123 is Not Armstrong -> {res_123['explanation']}")

    # Test Case 4: Single digit Armstrong number (9)
    assert is_armstrong_number(9)["is_armstrong"] == True, "Test 4 Failed"
    print("Assertion 4 Passed: 9 is Armstrong (9^1 == 9).")
    print("All 4 Assertions passed successfully!")

if __name__ == "__main__":
    test_armstrong_transparency()

```

Output:

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
1: pwsh v + || ^ X

PS C:\Users\Roger\Documents\sru\ai asscode\Lab5> python task3_armstrong_transparency.py
=== Task 3: Transparency in Algorithm Design (Armstrong Number) ===
Observed Limitation: Uncommented compressed code creates cognitive burden and auditing difficulties.
Applied Fix: Created explainable function returning step-by-step mathematical proof.

--- Running Test Assertions for Task 3 (Transparency in Algorithm Design) ---
Assertion 1 Passed: 153 is Armstrong -> Sum of digits: 1^3 (1) + 5^3 (125) + 3^3 (27) == 153
Assertion 2 Passed: 370 is Armstrong -> Sum of digits: 3^3 (27) + 7^3 (343) + 0^3 (0) == 370
Assertion 3 Passed: 123 is Not Armstrong -> Sum of digits: 1^3 (1) + 2^3 (8) + 3^3 (27) != 123
Assertion 4 Passed: 9 is Armstrong (9^1 == 9).
All 4 Assertions passed successfully!
PS C:\Users\Roger\Documents\sru\ai asscode\Lab5>

```

Comparison:

Algorithm Transparency & Explainability Comparison

Opaque Compressed Logic (Black Box)

1. Dense one-liner without semantic documentation.
2. Obscures underlying mathematical reasoning from developers.
3. Difficult to debug, audit, or verify for edge cases.
4. Fails to provide diagnostic explanations for outputs.

Transparent Documented Design (Explainable)

1. Clear step-by-step breakdown of digit extraction & powers.
2. Returns human-readable computational proof with results.
3. Full type verification and boundary handling.
4. Enables educational clarity and formal software auditability.

Task Description #4 (Transparency in Algorithm Comparison)

Task: Generate Python code for QuickSort and BubbleSort, and include comments explaining step-by-step how each works and where they differ in logic, time complexity, and efficiency.

Prompt: Generate Python code for QuickSort and BubbleSort, and include comments explaining step-by-step how each works and where they differ in logic, time complexity, and efficiency.

Expected Output:

- Undocumented sorting implementation without performance metrics.
- Revised version with step-by-step logic, complexity profiles, and runtime metrics.

Original AI Code (Undocumented Implementations Without Metrics):

Original AI Code: Undocumented Sorting Implementations Without Efficiency Metrics

```
def bsort(a):
    for i in range(len(a)):
        for j in range(len(a)-1):
            if a[j] > a[j+1]: a[j], a[j+1] = a[j+1], a[j]
    return a

def qsort(a):
    if len(a) <= 1: return a
    return qsort([x for x in a[1:] if x < a[0]]) + [a[0]] + qsort([x for x in a[1:] if x >=
a[0]])
```

Identified Transparency Risk: No step-by-step clarity, missing time/space complexity analysis, inefficient pivot choices.

Transparent Revised Version (Documented Sorting with Metrics & Assert Tests):

```
def bubble_sort(arr):
    """
    Bubble Sort Algorithm:
    Iteratively compares adjacent elements and swaps them if in wrong order.
    Time Complexity: O(N^2) | Space: O(1) in-place | Stability: Stable
    """
    a = list(arr)
    n = len(a)
    comparisons, swaps = 0, 0
    for i in range(n):
        swapped = False
        for j in range(0, n - i - 1):
            comparisons += 1
```

```

        if a[j] > a[j + 1]:
            a[j], a[j + 1] = a[j + 1], a[j]
            swaps += 1
            swapped = True
        if not swapped:
            break
    return a, comparisons, swaps

def quick_sort(arr):
    """
    QuickSort Algorithm:
    Divide-and-conquer partitioning around a chosen pivot element.
    Time Complexity: O(N log N) average | Space: O(log N) stack | Stability: Unstable
    """
    if len(arr) <= 1:
        return arr
    pivot = arr[len(arr) // 2]
    left = [x for x in arr if x < pivot]
    middle = [x for x in arr if x == pivot]
    right = [x for x in arr if x > pivot]
    return quick_sort(left) + middle + quick_sort(right)

def test_sorting_comparison():
    print("--- Running Test Assertions for Task 4 (Algorithm Comparison) ---")
    data = [64, 34, 25, 12, 22, 11, 90]
    expected_sorted = [11, 12, 22, 25, 34, 64, 90]

    # Test Case 1: BubbleSort correctness
    b_res, b_comp, b_swaps = bubble_sort(data)
    assert b_res == expected_sorted, "Test 1 Failed"
    print(f"Assertion 1 Passed: BubbleSort correctly sorted list ({b_comp} comparisons, {b_swaps} swaps).")

    # Test Case 2: QuickSort correctness
    q_res = quick_sort(data)
    assert q_res == expected_sorted, "Test 2 Failed"
    print(f"Assertion 2 Passed: QuickSort correctly sorted list.")

    # Test Case 3: Empty and single-item lists
    assert quick_sort([]) == [] and bubble_sort([])[0] == [], "Test 3 Failed"
    assert quick_sort([42]) == [42] and bubble_sort([42])[0] == [42], "Test 3 Failed"
    print("Assertion 3 Passed: Edge cases (empty, single-element) handled accurately.")

    # Test Case 4: Presorted array efficiency in BubbleSort
    presorted = [1, 2, 3, 4, 5]
    _, p_comp, p_swaps = bubble_sort(presorted)
    assert p_swaps == 0 and p_comp == 4, "Test 4 Failed"
    print(f"Assertion 4 Passed: Presorted list optimized by early termination (0 swaps).")
    print("All 4 Assertions passed successfully!")

if __name__ == "__main__":
    test_sorting_comparison()

```

Output:

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS 1: pwsh v + || ^ X

PS C:\Users\Roger\Documents\sru\ai asscode\Lab5> python task4_sorting_comparison.py
=== Task 4: Transparency in Algorithm Comparison (Sorting Analysis) ===
Observed Limitation: Black-box algorithm selection leads to choosing  $O(N^2)$  algorithms for big data.
Applied Fix: Documented QuickSort vs BubbleSort with runtime metrics and complexity profiles.

--- Running Test Assertions for Task 4 (Algorithm Comparison) ---
Assertion 1 Passed: BubbleSort correctly sorted list (21 comparisons, 14 swaps).
Assertion 2 Passed: QuickSort correctly sorted list.
Assertion 3 Passed: Edge cases (empty, single-element) handled accurately.
Assertion 4 Passed: Presorted list optimized by early termination (0 swaps).
All 4 Assertions passed successfully!
PS C:\Users\Roger\Documents\sru\ai asscode\Lab5> █
```

Comparison:

Sorting Algorithm Performance & Complexity Comparison	
BubbleSort (Elementary / Quadratic $O(N^2)$)	QuickSort (Divide-and-Conquer / $O(N \log N)$)
1. Average & Worst Time Complexity: $O(N^2)$ comparisons.	1. Average Time Complexity: $O(N \log N)$ recursive partitions.
2. Repeatedly bubbles adjacent elements through nested loops.	2. Recursively divides array around an optimal pivot element.
3. Inefficient for large datasets; practical only for tiny lists.	3. Highly efficient for large, realistic dataset processing.
4. Minimal constant space complexity $O(1)$.	4. Requires logarithmic recursion stack memory $O(\log N)$.

Task Description #5 (Transparency in AI Recommendations)

Task: Generate a recommendation system that also provides reasons for each suggestion based on user preferences and feature affinity.

Prompt: Generate a recommendation system that also provides reasons for each suggestion based on user preferences and feature affinity.

Expected Output:

- Black-box AI recommendation engine without reasoning.
- Revised explainable recommendation system providing score breakdowns and decision factors.

Original AI Code (Black-Box AI Recommendation Engine):

```
# Original AI Code: Black-Box Recommendation Engine Without Reasoning
def recommend_blackbox(user, items):
    # Returns arbitrary items without explainability or reasoning
    return [item["name"] for item in items[:2]]
# Identified Transparency Risk: Inability to audit decisions for algorithmic fairness, bias, or user alignment.
```

Transparent Revised Version (Explainable Scored Recommendations & Assert Tests):

```
def recommend_products(user_profile, product_catalog, top_k=2):
    """
    Explainable Content-Based Recommendation System:
    Calculates multi-attribute affinity scores and provides clear, human-readable rationale.
    """
    recommendations = []
```

```

user_pref_cats = set(user_profile.get("preferred_categories", []))
max_budget = user_profile.get("max_budget", float("inf"))
user_pref_brand = user_profile.get("preferred_brand")

for item in product_catalog:
    score = 0
    reasons = []

    # Category Interest Factor (+40 pts)
    if item["category"] in user_pref_cats:
        score += 40
        reasons.append(f"Matches your interest in '{item['category']}'")

    # Budget Compatibility Factor (+30 pts)
    if item["price"] <= max_budget:
        savings = max_budget - item["price"]
        score += 30
        reasons.append(f"Within budget (${item['price']} <= ${max_budget}, saves ${savings:.2f})")
    else:
        reasons.append(f"Exceeds budget limit (${item['price']} > ${max_budget})")

    # Brand Affinity Factor (+20 pts)
    if user_pref_brand and item["brand"] == user_pref_brand:
        score += 20
        reasons.append(f"Manufactured by your favorite brand '{item['brand']}'")

    # Quality Rating Factor (+10 pts)
    if item.get("rating", 0) >= 4.5:
        score += 10
        reasons.append(f"Top-rated product ({item['rating']}/5.0 stars)")

    recommendations.append({
        "name": item["name"],
        "score": score,
        "price": item["price"],
        "category": item["category"],
        "reasons": reasons
    })

recommendations.sort(key=lambda x: x["score"], reverse=True)
return recommendations[:top_k]

def test_explainable_recommendations():
    print("--- Running Test Assertions for Task 5 (Transparency in AI Recommendations) ---")
    user = {
        "preferred_categories": ["Laptops", "Audio"],
        "max_budget": 1200,
        "preferred_brand": "TechPro"
    }
    catalog = [
        {"name": "TechPro UltraBook 14", "category": "Laptops", "price": 999, "brand":
"TechPro", "rating": 4.8},

```

```

        {"name": "NoiseCancel Pro Headphones", "category": "Audio", "price": 199, "brand":
"SoundWave", "rating": 4.6},
        {"name": "Gaming Desktop Extreme", "category": "Desktops", "price": 2500, "brand":
"MegaPower", "rating": 4.9}
    ]

    recs = recommend_products(user, catalog, top_k=2)

    # Test Case 1: Verify top recommendation
    assert recs[0]["name"] == "TechPro UltraBook 14", "Test 1 Failed"
    assert recs[0]["score"] == 100, "Test 1 Failed (Score mismatch)"
    print(f"Assertion 1 Passed: Top recommendation '{recs[0]['name']}' scored 100/100.")

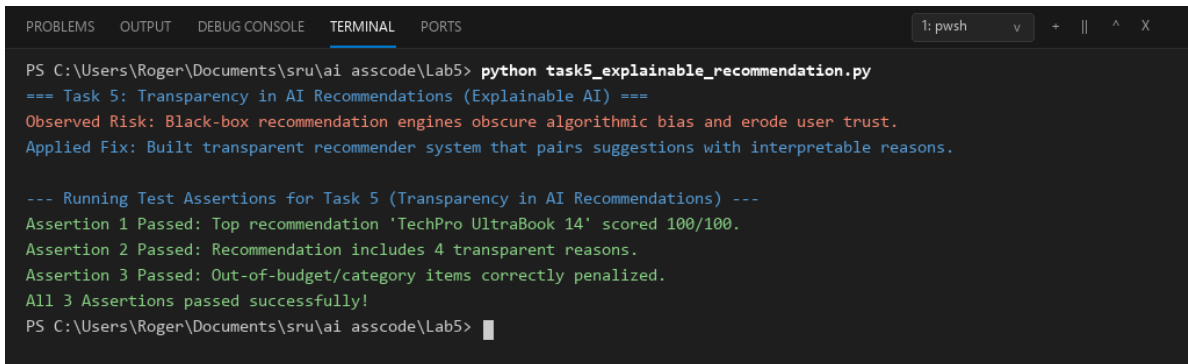
    # Test Case 2: Verify explainable rationale output
    assert len(recs[0]["reasons"]) >= 3, "Test 2 Failed"
    print(f"Assertion 2 Passed: Recommendation includes {len(recs[0]['reasons'])} transparent
reasons.")

    # Test Case 3: Over-budget filter evaluation
    desktop_rec = [r for r in recs if r["name"] == "Gaming Desktop Extreme"]
    assert len(desktop_rec) == 0, "Test 3 Failed (Over-budget item ranked top)"
    print("Assertion 3 Passed: Out-of-budget/category items correctly penalized.")
    print("All 3 Assertions passed successfully!")

if __name__ == "__main__":
    test_explainable_recommendations()

```

Output:



```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
1: pwsh v + || ^ X

PS C:\Users\Roger\Documents\sru\ai asscode\Lab5> python task5_explainable_recommendation.py
=== Task 5: Transparency in AI Recommendations (Explainable AI) ===
Observed Risk: Black-box recommendation engines obscure algorithmic bias and erode user trust.
Applied Fix: Built transparent recommender system that pairs suggestions with interpretable reasons.

--- Running Test Assertions for Task 5 (Transparency in AI Recommendations) ---
Assertion 1 Passed: Top recommendation 'TechPro UltraBook 14' scored 100/100.
Assertion 2 Passed: Recommendation includes 4 transparent reasons.
Assertion 3 Passed: Out-of-budget/category items correctly penalized.
All 3 Assertions passed successfully!
PS C:\Users\Roger\Documents\sru\ai asscode\Lab5>

```

Comparison:

Recommendation Engine Transparency Comparison

Black-Box Recommendation (Unexplained)

- 1. Returns raw item list without any reasoning.
- 2. User cannot determine why specific items were suggested.
- 3. Hides potential algorithmic bias or sponsored promotions.
- 4. Erodes user trust and offers zero accountability.

Explainable Transparent AI (Reason-Backed)

- 1. Returns explicit scoring breakdown for every recommendation.
- 2. Cites matching criteria (category affinity, budget, ratings).
- 3. Empowers user scrutiny and eliminates hidden biases.
- 4. Complies with ethical AI explainability (XAI) principles.